

Resolution Time Violation Prediction for Broadband Internet Customer Problems

Ümit Kanoğlu
Computer Engineering
Ozyegin University
İstanbul, Türkiye
umit.kanoglu@ozu.edu.tr

Reyhan Aydoğan
Computer Engineering
Ozyegin University
İstanbul, Türkiye
reyhan.aydogan@ozyegin.edu.tr

Olca Taner Yıldız
Computer Engineering
Ozyegin University
İstanbul, Türkiye
olcay.yildiz@ozyegin.edu.tr

Abstract—Nowadays, we have come across more and more machine learning techniques being applied to provide intelligent solutions for all sorts of problems in our lives. Machine learning also finds a wide application in the telecommunication domain. The problems that customers encounter in the telecommunication domain such as slow Internet speed, connection problems must be resolved within a given certain time interval; otherwise, the company has to pay a penalty due to the service level agreement. Accordingly, this work focuses on detecting any potential resolution time validation cases and aims to predict such violation cases in advance. We worked on real customer problem data and compare the performance of eleven different machine learning algorithms. Moreover, we analyze what data types play a key role in detecting violations. Furthermore, we investigate whether ensemble techniques such as voting, blending, and stacking can improve prediction performance. We achieve F2 scores higher than 80% with ensemble techniques.

Keywords—SLA violation prediction, Internet problems, classification, machine learning, telecommunications

I. INTRODUCTION

Telecommunication companies operate in a highly competitive market, where service quality is ensured through Service Level Agreements (SLAs) with customers [1]. These agreements define specific timeframes for resolving issues such as broadband internet problems. The increasing reliance on home internet especially during the COVID-19 pandemic [2] has made timely resolution even more critical, as SLA violations may lead to penalties and reduced service availability.

Machine learning (ML) is widely used in the telecom industry to support strategic decisions. Examples include churn prediction [3], customer income estimation [4], and fault detection in access networks [5]. Hani et al. [6], for instance, use Support Vector Regression to forecast SLA violations in cloud-based services. While these studies address service quality or marketing strategies, few focus on predicting resolution time violations at the moment a customer reports a problem.

This study aims to fill that gap by predicting SLA resolution violations for internet issues using real world data. The dataset includes over 80,000 customer reported problems with 11 core features. We apply feature engineering and evaluate 11 ML models, including Gaussian Naive Bayes (GNB), CatBoost Classifier (CB), Light Gradient Boosting Machine (LGBM), Extreme Gradient Boosting (XGB),

Histogram-Based Gradient Boosting (HGB), Multilayer Perceptron (MLP), Random Forest Classifier (RFC), Support Vector Machine (SVM), K-Neighbors Classifier(KNN), Logistic Regression (LR) and Perceptron (P) [7]. We also compare their performance across different feature sets raw, one hot encoded, and high importance. Inspired by findings in [8], we further explore ensemble methods like voting, stacking, and blending to improve performance.

The remainder of the paper is structured as follows: Section II reviews related work; Section III-A describes the dataset and preprocessing steps; Section III-B and Section III-C explains the prediction models; Section IV presents performance comparisons; and Section V concludes the paper.

II. RELATED WORK

Recent ML studies have introduced proactive methods for SLA assurance in telecommunications. Tuna and Soysal [9] proposed a multi-step traffic prediction framework with an SLA-aware loss function, improving prediction accuracy in Next-G networks. Gudepu et al. [10] used generative AI to optimize QoS model retraining in Beyond 5G and Open RAN, while a drift-handling framework [11] enabled real-time model adaptation for SLA compliance at the edge.

Traditional studies often focus on cloud or SDN-based services using virtual environment data for QoS or QoE prediction. Ahmed et al. [12] used system metrics to forecast FPS and ABS violations in VOD services. Bendriss et al. [13] predicted VOIP SLO breaches using ANN and NFV-based features. Terra et al. [14] employed XAI methods like SHAP and LIME to detect SLA latency issues in a simulated 5G environment. Although informative, their reliance on synthetic data limits real-world applicability.

Roychowdhury et al. [15] predicted SLO compliance based on evolving ticket status using logistic regression, updating every 15 minutes. In contrast, we make predictions at the ticket's creation time. Oralhan [16] applied various classifiers to predict problem resolution, but their dataset spanned only three months and lacked ensemble analysis.

This study utilizes one year of real-world broadband incident data to predict SLA violations at the time of issue reporting, evaluating 11 machine learning algorithms and three ensemble techniques. To our knowledge, this is the first

application of such a comprehensive approach in this domain.

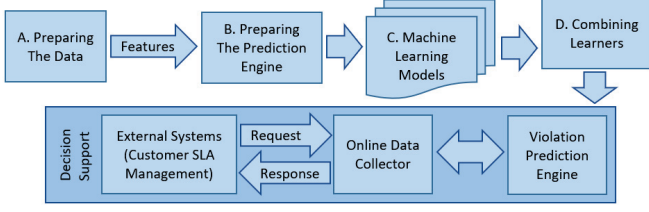


Fig. 1. Prediction of The SLA Violation.

III. PREDICTION OF THE SLA VIOLATION

In telecommunications, customer issues must be resolved within a specified timeframe defined by a SLA. Failure to meet this time may result in penalties, such as compensating customers, which leads to revenue loss. Predicting such violations in advance allows solution teams to act more proactively and meet SLA targets. For this, we develop a ML based model to predict SLA violations when problems are first reported.

Customer issues are logged through web portals or call centers and managed via a Customer Problem Management system [17]. Our model uses one year of real data from this system. As shown in Fig. 1, we preprocess the data, engineer relevant features, and evaluate multiple ML algorithms. The best-performing model serves as the core of our prediction engine.

We integrate this prediction engine into a decision support model that connects with Customer Problem Management, SLA/Jeopardy Management [18], and Workforce Management systems. Once a problem is reported, features are extracted and passed to the engine, which estimates SLA violation risk. If a risk is detected based on defined thresholds, solution teams are alerted to take early action.

A. Preparing The Data

In the study, we use real data by anonymizing it. We collect the data of customer problems for one year from the real environment. The data consists of 80K instances, where each instance includes 11 features. Table I consists of the features obtained from the problems reported by the customers. The 11 basic features we obtained from the core dataset are *CFS_Code*, *CFS_Plan_Code*, *Opening_Time*, *Closing_Time*, *Duration*, *Directorate*, *City*, *Region*, *Priority*, *Extra_Operation*, *Robotic Process Automation (RPA)*. We applied one-hot encoding to categorical features. The *Hour*, *Day*, *Month*, *Quarter* and *Shift_Period* features are derived from *Opening_Time*. According to the data distribution, the number of problem notifications is high between 9 and 12 o'clock, it is taken as *Peak_Time*. Since the solution teams work mobile, the time between 11 and 14 o'clock is the *Lunch_Break* period. Saturday and Sunday are *Weekend*. Problems occurring in September are considered as the *Back_to_School* period. *Istanbul* is also marked as it is the most crowded city in Turkey. Cities with a population of more than 3M are marked as *Metropol*. In addition, the *City_Population* feature is created according to the popu-

TABLE I. ALL FEATURES IN THE DATASET

Features	Description
CFS_Code(H)	Customer Facing Service (CFS) codes
CFS_Plan_Code(H)	define service types
Opening_Time	The time period in which the problem occurred
Closing_Time	
Duration	Categorical features that define the location
Directorate	
City	The priority of the problem. 1-Very Low...5-Urgent
Region(H)	
Priority(H)	Extra operation or RPA applied? True, False
Extra_Operation, RPA	
Hour, Opening_Day(H), Month	Extracting from <i>Opening_Time</i>
Shift_Period(H)	
Season(H)	Extracting from <i>Opening_Time</i> . 1-Spring, 2-Summer, 3-Fall, 4-Winter
Quarter(H)	
Back_to_School(H)	Quarters of the year, extracting from <i>Opening_Time</i>
Peak_Time	
Lunch_Break	If September 1 else 0
Weekend	
City_Population(H)	0-less 500K, 1-over 500K, 2-over 1M, 3-over 2M, 4-over 3M, 5-over 4M, 6-over 5M, 7-over 6M
Metropol	
Istanbul	If <i>City_Population</i> over 3M 1 else 0
Violation	
	Target value. If solution duration more than 48 hours 1 else 0

lation. *Violation* is the target value to be predicted. If the customer problem is not resolved within 48 hours, it takes the value 1-Violation, otherwise, it is 0-Successful. Additionally, the features with high (H) importance are shown in Table I. These features are marked by using random forest classifier [7] high importance values and logistic regression coefficient values.

We analyzed model performance using three datasets obtained from the features listed in Table I. All the features listed in the table and the high importance features (H), have been applied with one-hot encoding to form our first two datasets. We also obtained our third dataset without applying one-hot encoding to categorical features.

B. Preparing the Prediction Engine

Various libraries have been developed in different programming languages for ML. Python is one of these programming languages. We used the scikit-learn library in Python [25]. We also used Pandas and Numpy libraries for organizing data and mathematical operations. In our study, we preferred 11 different classification algorithms that are widely used. These algorithms are GNB, CB, LGBM, XGB, HGB, MP, RFC, SVM, KNN, LR, and Perceptron. We trained the algorithms with real data and created the models with supervised learning [7].

One of the algorithms we used is GNB. It is useful while working with continuous values whose probabilities, means, and variances can be modeled using Gaussian distributions associated with each particular class [7]. Class membership probabilities are used to estimate the probability of belonging to a particular class [19]. Bayesian classification is based

on the Bayes' theorem [20]. Gradient Boosting is an ensemble learning technique for regression and classification, combining Gradient Descent and Boosting. CatBoost, LightGBM, XGBoost, and Histogram-Based Gradient Boosting are popular implementations. XGBoost controls overfitting through regularization [21], while LightGBM and HGB use histogram-based binning and leaf-wise tree growth to improve speed and scalability [7]. CatBoost, effective for categorical features, also relies on binary decision trees [22].

TABLE II. PARAMETERS OF FORMULAS

	Predicted as Positive	Predicted as Negative
Actually Positive	True Positive (TP)	False Negative (FN)
Actually Negative	False Positive (FP)	True Negative (TN)

An Artificial Neural Network (ANN) is a simulation of the interconnections formed in the human brain. A typical ANN architecture has inputs, outputs, and a bias. ANN architecture differs greatly by layer. The most common and simple architecture is a Perceptron with two inputs, a hidden layer, and a single output. Neural networks are mostly backpropagated to be used for classification and prediction [22]. The Perceptron algorithm is made more complex by increasing the hidden layer between input and output. Thus, the Multilayer Perceptron algorithm is created [7]. Decision trees are one of the supervised learning algorithms that can be applied to both classification and regression problems. The Random Forest algorithm is a well-known tree-based ensemble learning method. The Random Forest differs from other standard trees in that each node is randomly split among a subset of predictors using the best importance. The Random Forest is used for both classification and regression [22]. SVM is a supervised learning technique often used in remote sensing research. The Support Vector algorithm works by finding the decision boundary. The algorithm tries to find the optimum hyperplane boundary through multiple iterations to better distinguish the patterns in the training data. When these patterns are found, the algorithm applies them to the evaluation dataset to check for accuracy [21]. The other two algorithms used in the study are KNeighbors and Logistic Regression. In particular, KNeighbors is a technique that can be used for different purposes and is based on the idea that samples close to a predefined distance metric are also similar so they can share their unique properties [22]. Logistic Regression as a statistical method is used to analyze a dataset containing the independent variables that determine an outcome. The outcome is measured with binary variables. Binary variable represents two different results as 0 and 1 [23].

We also use ensemble methods in the study. The predictions of the algorithms are combined with the voting ensemble solution. The ensemble is made in two different ways as hard and soft. With hard voting, the one with the largest number of predictions is used. Soft voting with the probability vectors for each predicted class for all classifiers are summed up [22]. In addition, in the study, we create new features set with the predictions of 11 different algorithms with blending and stacking [22]. New models are created for 11 different algorithms by using the features set consisting of the results. Thus, We make a two-stage prediction.

We compared algorithm performances with Accuracy, Precision, Recall, F1, and F2 scores. Formulas according to the definitions in Table II are as listed in equations (1) to (5) [24]. Accurate detection of violations is important in our study. Therefore, violation recall values have been taken into account so the F2 value with high recall weight is used when comparing the performances of the studied algorithms.

$$\text{Accuracy} = (|TP| + |TN|) / (|TP| + |FP| + |TN| + |FN|), \quad (1)$$

$$\text{Precision} = |TP| / (|TP| + |FP|), \quad (2)$$

$$\text{Recall} = |TP| / (|TP| + |FN|), \quad (3)$$

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (4)$$

$$F_2 = 5 \times \text{Precision} \times \text{Recall} / (4 \times \text{Precision} + \text{Recall}). \quad (5)$$

In the study, first of all, we tried to tune the hyperparameters that show the best performance of the algorithms. We used important hyperparameters for all algorithms. Grid and randomized search were used for this process. All possible values of hyperparameters were tried with grid search. Different values are tried as much as the number determined by randomized search. Possible hyperparameter values for each algorithm, search method, and the best hyperparameters values found are shown in Appendix A. Appendix A also shows the best hyperparameters search duration. We apply ensemble classification using models determined by the grid and randomized search using all features with 10Fold cross-validation [7]. Voting Classifier from the scikit-learn library was used. With stacking and blending, new models were created for each algorithm. Finally, we divide into the data set four parts according to the shift period feature, the applied methods are repeated for each part and the results are compared. We obtain and compare 485 different models with different features and different methods with 11 different ML algorithms. Details of the applied methods are given in the following subsections.

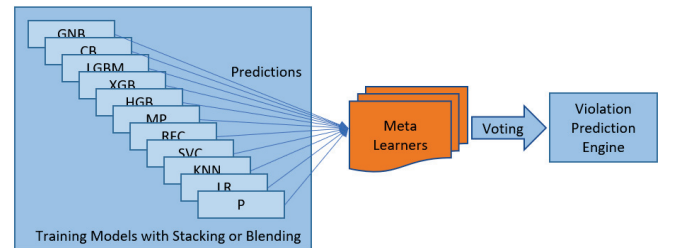


Fig. 2. Combining Learners for SLA Violation Predictions.

C. Machine Learning Models

We created different models by using ML algorithms in the study. First of all, we train each algorithm with the feature sets shown in Table I. Algorithms are combined with hard

and soft voting. We improve the performances of the models by stacking, blending, and data splitting. Details about the models created are given in the following subsections.

1) All Algorithms with All Features

Within the scope of the study, We use 11 different algorithms. As seen in Table III, these algorithms are; GNB, CB, LGBM, XGB, HGB, MP, RFC, SVM, KNN, LR, Perceptron [7]. Algorithms are ordered from largest to smallest according to the best F2 score in Table III. Firstly, We create the models with all features for all algorithms and we compare the performances. As a result, the best F2 score is obtained with GNB. However, it should be noted that the accuracy and precision values for GNB are relatively low. Then the best scores are obtained with the boosting algorithms CB, LGBM, XGB, and HGB. MP algorithm achieves the sixth-best score. The performances of LR and Perceptron are worse than RFC, SVM, and KNN classifiers. Performance scores for algorithms other than GNB are balanced. For this reason, we try to balance the performance of the GNB algorithm and improve the performance of other algorithms by applying different methods.

In the next sections, we compare the performances of the algorithms in the different methods applied.

TABLE III. ALL MODELS WITH ALL FEATURES

Model	Features	Accuracy	Precision	Recall	F1	F2
GNB	All	0,55	0,52	0,91	0,66	0,79
CB	All	0,67	0,68	0,62	0,65	0,63
LGBM	All	0,67	0,68	0,62	0,65	0,63
XGB	All	0,67	0,67	0,62	0,64	0,63
HGB	All	0,67	0,68	0,61	0,64	0,62
MP	All	0,65	0,66	0,61	0,63	0,62
RFC	All	0,65	0,67	0,58	0,62	0,60
SVM	All	0,64	0,65	0,58	0,61	0,59
KNN	All	0,59	0,59	0,59	0,59	0,59
LR	All	0,61	0,61	0,55	0,58	0,56
P	All	0,57	0,56	0,55	0,56	0,55

2) All Algorithms with High Importance and without One Hot Encoding Features

We create new models with features with high importance and features without one hot encoding after the models are obtained by applying all features for all algorithms. High Importance (High) features are obtained using Random Forest and Logistic Regression algorithms. All features used without one hot encoding (non-OHE) are Day, Month, Season, Quarter, and Shift Period. The new results are shown in Table IV. Apart from all the features, there is no significant difference for the High and non-OHE feature sets. However, the F2 and Recall scores for GNB are getting better, but looking at all the scores, the performance metrics are not balanced. Although accuracy and precision scores are about 0.5, recall scores are up to 0.98.

IV. RESULTS OF COMBINING LEARNERS FOR SLA VIOLATION PREDICTIONS

In our study, we do not only compare the performances of algorithms individually, but we also combine algorithm performances with ensemble methods. We use voting, stack-

TABLE IV. ALL MODELS WITH ALL FEATURES WITHOUT ONE HOT ENCODING AND HIGH IMPORTANCE FEATURES

Model	Features	Accuracy	Precision	Recall	F1	F2
GNB	High	0,51	0,5	0,98	0,66	0,82
LGBM	High	0,67	0,67	0,62	0,65	0,63
CB	High	0,67	0,68	0,61	0,64	0,62
MLP	High	0,66	0,67	0,61	0,64	0,62
XGB	High	0,66	0,67	0,61	0,64	0,62
HGB	High	0,66	0,68	0,6	0,64	0,61
RFC	High	0,66	0,67	0,6	0,63	0,61
SVM	High	0,65	0,66	0,6	0,63	0,61
KNN	High	0,6	0,59	0,59	0,59	0,59
P	High	0,55	0,54	0,59	0,56	0,58
LR	High	0,6	0,61	0,54	0,57	0,55
GNB	non-OHE	0,5	0,5	0,99	0,66	0,83
CB	non-OHE	0,67	0,68	0,62	0,64	0,63
XGB	non-OHE	0,67	0,68	0,62	0,65	0,63
LGBM	non-OHE	0,67	0,67	0,62	0,65	0,63
MLP	non-OHE	0,66	0,67	0,62	0,64	0,63
HGB	non-OHE	0,67	0,68	0,6	0,64	0,61
RFC	non-OHE	0,66	0,67	0,6	0,64	0,61
SVM	non-OHE	0,65	0,66	0,6	0,63	0,61
P	non-OHE	0,57	0,55	0,61	0,58	0,60
KNN	non-OHE	0,6	0,59	0,58	0,59	0,58
LR	non-OHE	0,61	0,61	0,55	0,58	0,56

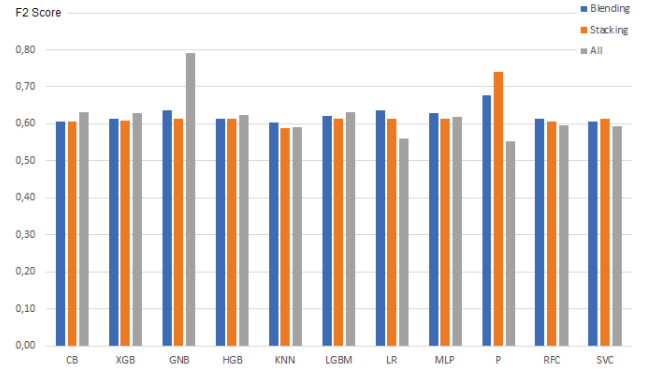


Fig. 3. F2 Scores for All Models with All Features after Stacking and Blending

ing, and blending ensemble techniques. With voting, the majority prediction made by more than one algorithm or the prediction with the highest probability determines the final prediction. In stacking and blending, the predictions made by the created models become the input of the meta learner algorithm. While models and meta learners are trained with the K-fold cross-validation technique in stacking, models and meta learners are trained by separating dataset train, validation, and test set in blending [26]. Fig. 2 shows the path followed when combining algorithm performances. First, the individual performance of the algorithms, detailed in the previous sections, are compared. Then, ensemble techniques are applied by making each algorithm a meta-learner, and voting, stacking, and blending performances are compared. Finally, the predictions of all algorithms after stacking and blending are again combined with soft and hard voting and compared.

For a training setup, all models by using stacking and blending are seen in Table III. We use every algorithm in the table as the stacking and blending algorithms. The result produced by each algorithm for all features is the input of the stacking and blending algorithms. F2 scores after stacking and blending for all features are in Fig. 3. A significant improvement in overall performance is seen in Perceptron, LR, and GNB algorithms. Perceptron F2 score after blending

is 0.13 higher. Perceptron F2 score after stacking is 0.19 higher. The F2 score is 0.05 higher for LR after stacking. After blending, the F2 score is 0.08 higher. For GNB, the F2 score has decreased, but the precision and recall values have increased, and the algorithm has been balanced with an acceptable F2 score.

For another training setup, all models by using stacking and blending are shown in Table IV. We create the models in Table IV using the high importance and without applying one hot encoding feature sets. The F2 scores after stacking and blending for the high importance feature set are in Fig. 4. Accordingly, significant improvement in overall performance is seen in Perceptron, LR, and GNB algorithms. Perceptron F2 score after stacking is 0.14 higher. After blending, the F2 score is very low and the model is out of balance. The F2 score is 0.06 higher for LR after stacking. After blending, the F2 score is 0.08 higher. As in all feature sets, the F2 score for GNB has decreased, but the precision and recall values have increased, and the algorithm has been balanced with an acceptable F2 score. The results after stacking and blending for features without applying one hot encoding are in Fig. 5. Accordingly, a significant improvement in overall performance is seen in Perceptron, LR, and GNB algorithms. Perceptron F2 score is 0.13 higher after stacking. After blending, the F2 score is 0.07 higher. The F2 score is 0.06 higher for LR after stacking. After blending, the F2 score is 0.08 higher. The F2 score increment for LR is the same for all feature sets. The F2 score for GNB has decreased again, but the precision and recall values have increased and the algorithm has been balanced with a good F2 score.

The best improvement in performance is achieved with the Perceptron algorithm by stacking. The prediction made by 11 different algorithms after stacking is used as input for the Perceptron algorithm. The model is created in two stages. The perceptron is trained with a simpler set of inputs. Algorithm performance improves significantly as seen in Fig. 3.

We also examine the results of the models by splitting the dataset. Algorithms are trained with 4 different datasets by splitting them according to shift period values. After splitting according to the shift period, the results of the voting algorithm are also improved. Relatively better values obtained after voting and splitting can be seen in Table V. We obtain better results for split datasets according to different shift period values in GNB, Perceptron, and LR algorithms. Stacking and blending are also shown in Table V. For the hard voting algorithm, the voting results obtained only for shift period 2 are relatively better. But when we consider the recall values, we see that the results are not balanced.

V. CONCLUSION AND FUTURE WORK

In this study, we analyzed broadband internet issues reported by customers to predict SLA resolution time violations using ML. We performed feature engineering to derive new insights from the raw data and tuned hyperparameters for 11 classification algorithms using both grid search and randomized search methods. Models were trained on three different feature sets: all features, all without one-hot encoding, and high-

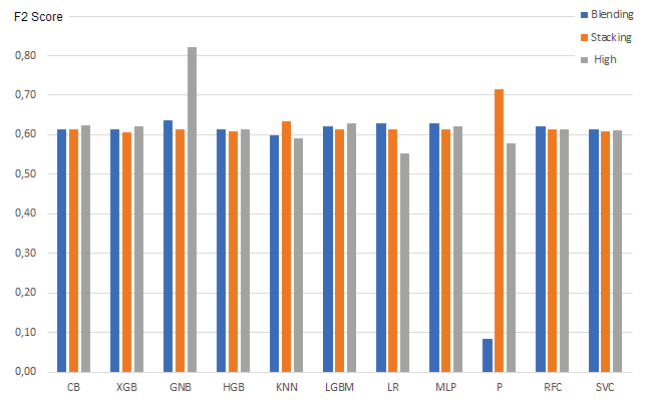


Fig. 4. F2 Scores for All Models with High Importance Features after Stacking and Blending.

TABLE V. EXAMPLE SCORES AFTER SPLIT DATA BY SHIFT PERIOD (SP) AND VOTING

Model	Features	Ensemble	Split	Accuracy	Precision	Recall	F1	F2
GNB	High	No	SP2	0.53	0.52	1	0.69	0.84
P	non-OHE	Blending	SP2	0.53	0.52	1	0.68	0.84
GNB	non-OHE	No	SP2	0.53	0.52	0.99	0.68	0.84
GNB	non-OHE	No	SP1	0.51	0.5	0.99	0.66	0.83
P	High	Blending	SP2	0.53	0.52	0.97	0.68	0.83
GNB	All	No	SP2	0.55	0.54	0.95	0.68	0.82
GNB	High	No	SP1	0.51	0.5	0.98	0.66	0.82
P	All	Blending	SP2	0.55	0.53	0.95	0.69	0.82
GNB	non-OHE	No	SP4	0.51	0.5	0.97	0.66	0.82
GNB	High	No	SP4	0.52	0.5	0.96	0.66	0.81
P	non-OHE	Stacking	SP1	0.52	0.51	0.95	0.66	0.81
GNB	All	No	SP1	0.54	0.52	0.93	0.67	0.80
P	All	Stacking	SP2	0.53	0.52	0.92	0.67	0.80
GNB	High	No	SP3	0.48	0.46	0.92	0.62	0.77
P	All	Stacking	SP1	0.58	0.55	0.85	0.67	0.77
GNB	All	No	SP4	0.56	0.52	0.84	0.64	0.75
P	All	Blending	SP1	0.59	0.56	0.75	0.64	0.70
P	High	Stacking	SP2	0.6	0.59	0.72	0.65	0.69
LR	High	Blending	SP2	0.64	0.64	0.7	0.67	0.69
LR	All	Blending	SP2	0.64	0.63	0.7	0.66	0.68
Voting-Hard	All	No	SP2	0.64	0.65	0.68	0.66	0.67
GNB	non-OHE	No	SP3	0.54	0.49	0.74	0.59	0.67
MLP	All	Blending	SP2	0.64	0.65	0.67	0.66	0.67
LR	non-OHE	Blending	SP2	0.64	0.65	0.67	0.66	0.67

importance features identified through feature importance analysis.

To enhance model performance, we applied ensemble techniques such as hard and soft voting, stacking, and blending. These were also applied to data subsets split by shift periods, resulting in a total of 585 models. Our findings indicate that individual algorithms alone did not achieve consistently high performance. While GNB produced high F2 and recall scores, its precision was significantly lower, making it less balanced overall. Simple voting approaches did not significantly improve outcomes. In contrast, stacking and blending led to substantial performance gains particularly for basic models like Perceptron and Logistic Regression, which showed significant increases in both F2 and recall scores after

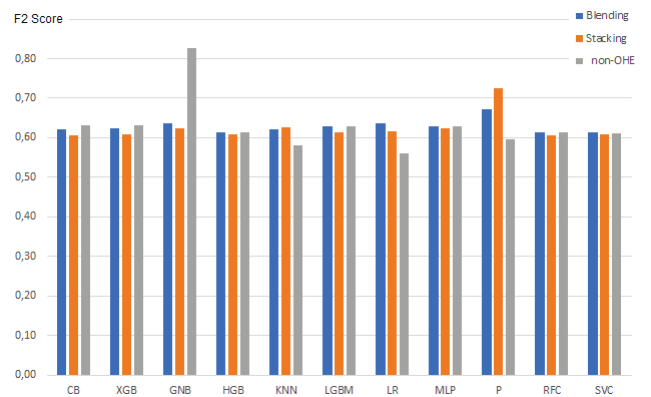


Fig. 5. F2 Scores for All Models with All Features without One Hot Encoding after Stacking and Blending.

ensembling.

Based on these findings, we developed a decision support model that incorporates the best-performing ensemble model. This model is designed to work alongside systems such as Customer Problem Management, SLA Jeopardy Management, and Workforce Management. When a customer issue is reported, the prediction engine analyzes relevant features in real time and estimates the risk of SLA violation. If a risk is detected, the system alerts resolution teams so they can prioritize the issue. This proactive approach has the potential to reduce SLA breaches and minimize the financial penalties incurred by the company.

This study makes a meaningful contribution to SLA violation prediction in telecommunications, an area often overlooked in favor of availability focused SLA studies in cloud or SDN contexts. Our approach makes predictions at the time of ticket creation using actual service issue features. The methodology is generalizable and can be applied in other domains where SLA adherence is critical. As future work, we aim to deploy the model in a live environment to monitor real time performance and further validate its effectiveness in operational settings.

APPENDIX A FINDING BEST HYPER PARAMETERS WITH ALL FEATURES.

Algorithm	Search Type	Duration	Best Hyperparameters
MLP	RandomizedSearch n_iter=50	37 Hours	max_iter = 2000 hidden_layer_sizes = (100,) activation = 'tanh' solver = 'sgd' alpha = 0.01 learning_rate = 'constant'
RFC	RandomizedSearch n_iter=50	6 Hours	criterion = 'entropy' n_estimators = 800 max_features = 'auto' max_depth = 10 min_samples_split = 5 min_samples_leaf = 1 bootstrap = True
LR	GridSearch	1 Hour	solver = 'liblinear' penalty = 'l1' C = 0.01
KNN	RandomizedSearch n_iter=50	10 Hours	n_neighbors = 19 metric = 'minkowski' weights = 'uniform' leaf_size = 30p = 2
SVM	RandomizedSearch n_iter=10	62 Hours	kernel = 'rbf' gamma = 'scale' C=1
P	GridSearch	2 Mns	eta0 = 0.01 max_iter = 1000
GNB	GridSearch	2 Mns	var_smoothing = 0.2
XGB	RandomizedSearch n_iter=50	18 Hours	n_estimators = 900 learning_rate = 0.06 subsample = 0.9 max_depth = 6 colsample_bytree = 0.5 min_child_weight = 1
LGBM	RandomizedSearch n_iter=50	1 Hour	boosting_type = 'dart' num_leaves = 7 max_depth = 49 learning_rate = 0.7 n_estimators = 251 class_weight = 'balanced' min_child_samples = 50 random_state = 0 importance_type = 'gain'
CB	RandomizedSearch n_iter=50	3 Hours	Default
HistGradientBoosting	RandomizedSearch n_iter=50	20 Mns	Default

REFERENCES

- [1] E. Marilly, O. Martinot, S. Betgé-Brezetz, and G. Delègue, "Requirements for service level agreement management," in *IEEE Workshop on IP Operations and Management*, 2002, pp. 57–62.
- [2] C. Zachreson, E. Martino, M. Tomko, F. M. Shearer, R. Bentley, and N. Geard, "Mapping home internet activity during COVID-19 lockdown to identify occupation related inequalities," *Sci. Rep.*, vol. 11, no. 1, pp. 1–13, 2021.
- [3] H. Li, D. Yang, L. Yang, Y. Lu, and X. Lin, "Supervised massive data analysis for telecommunication customer churn prediction," in *Proc. IEEE Int. Conf. BDCloud-SocialCom-SustainCom*, 2016, pp. 163–169.
- [4] P. Sundsøy, J. Bjelland, B.-A. Reme, A. M. Iqbal, and E. Jahani, "Deep learning applied to mobile phone data for individual income classification," in *Proc. Int. Conf. Artificial Intelligence: Technologies and Applications*, 2016, pp. 24–25.
- [5] F. LiLik, P. Simonyi, and L. T. Kóczy, "Computational intelligence in performance evaluation and fault prognosis in telecommunication access networks," *Tech. Transact.*, vol. 2013, no. 2 AC (10), 2014.
- [6] A. F. M. Hani, I. V. Paputungan, and M. F. Hassan, "Support vector regression for service level agreement violation prediction," in *Proc. IC3INA*, 2013, pp. 307–311.
- [7] G. Bonaccorso, *Machine Learning Algorithms: A Reference Guide to Popular Algorithms for Data Science and Machine Learning*. Packt Publishing, 2017.
- [8] O. Gungor, B. Aksanli, and R. Aydoğan, "Algorithm selection and combining multiple learners for residential energy prediction," *Future Gener. Comput. Syst.*, vol. 99, May 2019.
- [9] E. Tuna and A. Soysal, "Multivariate and multi-step traffic prediction for NextG networks with SLA violation constraints," *arXiv preprint arXiv:2304.11156*, Apr. 2023.
- [10] V. Gudepu, B. Chirumamilla, V. R. Chintapalli et al., "Generative-AI for AI/ML model adaptive retraining in Beyond 5G networks," *arXiv preprint arXiv:2408.14827*, Aug. 2024.
- [11] Authors, "The drift handling framework for open radio access networks," *Int. J. Antennas Propag.*, 2024.
- [12] J. Ahmed, A. Johnsson, R. Yanggratoke, J. Ardelius, C. Flinta, and R. Stadler, "Predicting SLA violations in real time using online machine learning," *arXiv preprint arXiv:1509.01386*, 2015.
- [13] J. Bendriss, I. G. B. Yahia, and D. Zeghlache, "Forecasting and anticipating SLO breaches in programmable networks," in *Proc. 20th Conf. Innovations in Clouds, Internet and Networks (ICIN)*, 2017, pp. 127–134.
- [14] A. Terra, R. Inam, S. Baskaran, P. Batista, I. Burdick, and E. Fersman, "Explainability methods for identifying root-cause of SLA violation prediction in 5G network," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, 2020, pp. 1–7.
- [15] S. Roychowdhury, A. Sarkar, and K. Bedhu, "Near-real time prediction of service requests' SLA breach," in *Int. Conf. Business Analytics and Intelligence*, 2015.
- [16] B. Oralhan, "Troubleshooting analysis in telecommunication sector using data mining approach," *Bus. Manag. Stud.: Int. J.*, vol. 8, no. 1, pp. 1026, 2020.
- [17] TM Forum, "TMF621 Trouble Ticket Management API REST Specification R19.0.1," Jan. 2021. [Online]. Available: <https://www.tmforum.org/resources/standard/tmf621-trouble-ticket-management-api-rest-specification-r19-0-0/>
- [18] TM Forum, "TMF623 SLA Management API REST Specification R14.5.1," Dec. 2019. [Online]. Available: <https://www.tmforum.org/resources/standard/tmf623-sla-management-api-rest-specification-r14-5-0/>
- [19] R. Theodorescu, "Good, I. J.: The estimation of probabilities. An essay on modern Bayesian methods," *Biometrische Zeitschrift*, vol. 10, pp. 87–87, Jan. 1968.
- [20] J. Hilden, "Statistical diagnosis based on conditional independence does not require it," *Comput. Biol. Med.*, vol. 14, pp. 429–435, Feb. 1984.
- [21] D. McCarty, K. H. Woo, and H. Lee, "Evaluation of light gradient boosted machine learning technique in large scale land use and land cover classification," *Environments*, vol. 7, Oct. 2020.
- [22] A. Ibrahim, R. Raheem, M. Muhammed, R. Abdulaziz, and S. Ganiyu, "Comparison of the CatBoost classifier with other machine learning methods," *Int. J. Adv. Comput. Sci. Appl.*, vol. 11, Dec. 2020.
- [23] J. Peng, K. Lee, and G. Ingersoll, "An introduction to logistic regression analysis and reporting," *J. Educ. Res.*, vol. 96, pp. 3–14, Sept. 2002.
- [24] J. Miao and W. Zhu, "Precision-recall curve (PRC) classification trees," *Evol. Intell.*, Apr. 2021.
- [25] Scikit-learn Developers, "Scikit-learn: Machine Learning in Python." [Online]. Available: <https://scikit-learn.org/stable/>
- [26] D. Sarkar and V. Natarajan, *Ensemble Machine Learning Cookbook: Over 35 Practical Recipes to Explore Ensemble Machine Learning Techniques Using Python*. Packt Publishing Ltd., 2019.